

# AI Real Estate Portfolio Analyst

## Engineering Assignment

---

|                |                                                                              |
|----------------|------------------------------------------------------------------------------|
| Timebox        | Approximately 1–2 days                                                       |
| What you build | A conversational AI portfolio analyst, plus a simple business-side interface |
| On submission  | A 5–10 minute walkthrough of your system                                     |
| Dataset        | <a href="#">Click here to download</a>                                       |
| Submission     | <a href="#">Click here to submit</a>                                         |

## 1 The Problem

---

Build a **two-way AI Real Estate Portfolio Analyst**.

A user should be able to have an ongoing conversation with an AI about their real-estate portfolio through a simple chat interface that **simulates WhatsApp**. The assistant should understand the user's portfolio, analyse it, answer questions, maintain context, and take appropriate actions.

**You do not need to integrate the WhatsApp API.** The chat interface is only a simulation of the WhatsApp experience.

## 2 What should the agent be able to do?

---

### 2.1 Understand the portfolio

A user can add or update properties through conversation.

*"I have a 5,000 sq ft retail property in Bandra worth ₹12 Cr. Add it to my portfolio."*

The agent should understand the information, ask for important missing information, and persist it.

### 2.2 Analyse the portfolio

The user should be able to ask questions such as:

*"What does my portfolio look like?"*

*"How much of my portfolio is retail?"*

*"Which property generates the most rent?"*

*"What is my total portfolio value?"*

*"Which properties do I own in Mumbai?"*

*"Compare my residential and commercial exposure."*

The agent should retrieve information from the backend and use appropriate tools and calculations rather than relying on information held only in the LLM context.

### 2.3 Have a natural conversation

The agent should maintain context across multiple messages.

*"Tell me about my retail properties."*

*"Which one is performing better?"*

*"What if I exclude the Bandra property?"*

*"How would that change my portfolio?"*

The system should distinguish between **actual portfolio data** and hypothetical analysis.

## 2.4 Take actions

Where appropriate, the agent should be able to modify the underlying data.

| *"Update the Bandra property's value to ₹14 Cr."*

The exact set of actions is up to you.

## 2.5 Business interface

Build a simple second interface for the business team to:

- See users and conversations.
- Inspect individual conversations.
- See relevant agent and tool activity.
- Identify conversations requiring attention.

Keep this simple. It is an **observation and control interface**, not a CRM.

## 3 Your Engineering Challenge

---

You have freedom over the architecture. We expect you to make decisions around:

- Agent and orchestration design.
- Model selection and model routing.
- Tool and function design.
- Conversation memory and state.
- Data storage and retrieval.
- Analytical calculations.
- Error handling and fallbacks.
- Observability.
- Conversation latency.
- Cost and scalability.

Use an agentic framework such as **LangChain or an equivalent framework**, and **OpenRouter or an equivalent model gateway**.

We are deliberately **not prescribing** whether you should use SQL, RAG, vector search, multiple agents, a single agent, workflows, queues, or anything else. Choose what makes sense and explain why.

## 4 Define the Agent

---

Create a `SOUL.md` describing the agent you designed. Use it to explain your decisions around:

- Role and purpose.
- Tone and voice.
- Behaviour and personality.
- Skills and capabilities.
- Tools available to the agent.
- How it handles uncertainty.
- What it should and should not do.
- How analytical conversations should work.
- When it should ask questions.
- When it should proactively surface information.
- When it should hand off to a human.

The specific personality and behaviour are up to you.

## 5 Performance Matters

---

Measure and document the **latency of your conversations**. At minimum, discuss:

- Typical end-to-end response time.
- Where time is spent.
- Model and tool latency.
- What you did to reduce unnecessary latency.
- What you would change if the system had to operate at significantly higher volume.

You should be able to explain why a response takes the amount of time it does.

## 6 The Dataset

A small synthetic dataset is provided — the download link is at the top of this document. It is intentionally small so you can focus on agent design, orchestration, state, tool use, reasoning, reliability and performance rather than data engineering.

| File                   | Contents                                                                                      |
|------------------------|-----------------------------------------------------------------------------------------------|
| users.csv              | 4 users, with city, stated preferences, preferred locations and a portfolio value preference. |
| properties.csv         | 12 properties across those 4 users, in Mumbai, Delhi NCR and Bengaluru.                       |
| sample_requests.csv    | 6 example requests, each with the capability and interpretation expected from the system.     |
| DATASET.md , README.md | The same data rendered as tables, plus notes on intent.                                       |

### 6.1 Property schema

| Column                                     | Notes                                                                                                                              |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| property_id , user_id                      | Identifiers. Each property belongs to exactly one user.                                                                            |
| property_type , sub_type                   | Retail, Commercial Office, Office, Residential; sub-types such as High-street retail, Mall retail, Apartment, Villa, Office floor. |
| location                                   | Free text, formatted as Locality, City . There is no separate city column.                                                         |
| area_sqft ,<br>current_estimated_value_inr | Populated for every row.                                                                                                           |
| purchase_price_inr                         | <b>Intentionally blank throughout.</b> Decide how your system handles missing historical cost.                                     |
| annual_rent_inr                            | Zero for vacant and self-occupied properties.                                                                                      |
| occupancy_status , tenant_status           | Tenanted, Vacant or Self-occupied.                                                                                                 |
| ownership_percent , status                 | 100 and Active for every row in the seed data.                                                                                     |

### 6.2 What the dataset supports

Total portfolio value; value by property type; retail, residential and office exposure; annual rental income; rental yield where value and rent are both present; highest and lowest value property; highest annual rent; geographic concentration; occupancy and vacancy; comparisons between property types; and hypothetical scenarios. These are examples, not a prescribed feature list.

**Read the data before you design around it.** Three things are worth noticing:

- Property type labels are not perfectly consistent. Decide how your system groups them, and say so.
- There are no dates anywhere, so time-based questions such as appreciation since purchase cannot be answered from the seed data. Handle that honestly rather than inventing an answer.
- sample\_requests.csv is a set of examples, not the complete set of supported queries. Do not build only to those six.

## 7 Data and External Access

You do **not** need access to any private property database, CRM, property portal, customer account or authenticated data source.

You may use publicly available APIs and technical documentation where required. Any external dependency should be clearly documented in your repository.

## 8 What to Submit

| Deliverable              | Detail                                                                                                                                                 |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Live interactive tool | A working application we can interact with ourselves.                                                                                                  |
| 2. GitHub repository     | Complete source code, setup instructions, architecture explanation, assumptions, key technical decisions, known limitations and external dependencies. |
| 3. SOUL.md               | Your definition of the agent's behaviour, voice, skills and boundaries.                                                                                |
| 4. Architecture diagram  | A simple explanation of how the system works end to end.                                                                                               |
| 5. Decision log          | The most important engineering decisions and trade-offs you made.                                                                                      |
| 6. Short walkthrough     | Be prepared to give a 5–10 minute walkthrough of the system and explain why you built it the way you did.                                              |

Submit everything through the submission form — [click here to submit](#). Check that your live app is reachable and your repository is accessible to us before you do.

## 9 Important

### **Use AI freely. We expect you to.**

The assignment is intentionally scoped to approximately **1–2 days**.

We are not testing how much code you can write without AI.

We are testing **how well you use AI, engineering principles and your own judgment to design, orchestrate and explain a working AI system.**